

Label Jaano — Backend Deep-Dive & Interview Prep

Every module, every function, every design decision — and how to defend them

Label Jaano · Smart India Hackathon

August 2026

Contents

How to use this guide	2
The thirty-second pitch (memorise this)	3
1. The big picture	3
1.1 What problem it solves	3
1.2 The one principle everything follows: <i>AI extracts, rules judge</i>	3
1.3 The three layers	4
1.4 The request lifecycle in one breath	4
2. The tech stack — every choice and why	4
2.1 Python 3 (standard library) for the engine	4
2.2 Dataclasses (not Pydantic) inside the engine	4
2.3 Rules as JSON “rule packs” (not code, not a database, not a trained model)	5
2.4 FastAPI + Uvicorn for the HTTP layer	5
2.5 Pydantic v2 at the API boundary	5
2.6 Google Gemini 2.0 Flash for structured extraction	6
2.7 PaddleOCR for word-level pixel geometry	6
2.8 OpenCV for reference-based calibration	6
2.9 pytest, plus bare-python3 test runners	6
2.10 The mock layer — the unsung hero	6
3. Architecture overview	7
3.1 The directory map	7
3.2 The scan-input contract — the seam	7
3.3 The eight rule packs and how they stack	8
3.4 Two tiers: scored vs reference-only	8
4. Deep code walkthrough	9
4.1 rule_engine/models.py — the vocabulary of the whole system	9
4.2 rule_engine/loader.py — loading and merging packs	10
4.3 rule_engine/checks.py — validators, symbols, conditions, and the dispatcher	11
4.4 rule_engine/engine.py — the judge	13
4.5 rule_engine/__init__.py — the public surface	15
4.6 The pipeline — backend/pipeline/	15
4.7 The API — backend/app/	17
4.8 The command-line runners	18
5. End-to-end trace: one photo to a verdict	18
6. The two-tier model, in depth	20
6.1 The problem	20

6.2 The design	20
6.3 Why the guarantee is <i>structural</i> , not a promise	20
6.4 How to say it in one line	20
7. Testing and verification	21
7.1 The test suites	21
7.2 The subset-assertion lesson (a real gotcha we fixed)	21
7.3 The verification philosophy	21
8. Extending, scaling, and the honest roadmap	21
8.1 How you extend it (three cases, increasing effort)	21
8.2 What's real today vs what's upstream	21
8.3 Scaling story (for the “will it scale?” question)	22
9. Interview & judging Q&A prep	22
9.1 Architecture & the core idea	22
9.2 The “why not just use AI / a trained model?” challenge	22
9.3 The rule engine internals	23
9.4 The pipeline, OCR, and the vision-LLM	24
9.5 Robustness and edge cases	24
9.6 The two-tier / reference standards	24
9.7 Legal accuracy & trust	25
9.8 Scaling, security, production	25
9.9 Curveballs & honest weaknesses	25
10. One-page cheat-sheet (the night before)	26
The one-liners (memorise verbatim)	26
Key numbers to have ready	26
The file map (backend/)	26
The 8 rule packs	27
The check types	27
Mini-glossary	27
If you blank on a hard question	28

How to use this guide

This is your single source of truth for the **backend** of Label Jaano. It has two halves:

1. **The deep code walkthrough** (Sections 1–8): the tech stack and *why* each piece was chosen, the architecture, then a file-by-file, function-by-function tour of the real code, and finally a full end-to-end trace of one photo becoming a verdict.
2. **The interview Q&A prep** (Section 9): the questions a judge or panel is most likely to ask, grouped by theme, each with a confident, accurate answer you can say out loud — including the hard “why not just use AI?” and “what are the weaknesses?” curveballs.

Read Sections 1–3 to get the mental model, skim Section 4 with the code open beside you, then drill Section 9 until the answers are automatic. Section 10 is a one-page cheat-sheet for the night before.

A note on honesty, which is itself a selling point: everywhere the code makes a *legal* claim, the rule stores the exact citation, and the regexes are engineered detectors, not the gazette text. When a judge asks “is this citation real?”, the honest, precise answer (“clause numbers are transcribed verbatim; the pattern that detects them is ours”) is stronger than a vague one.

The thirty-second pitch (memorise this)

Label Jaano checks whether a packaged-product label obeys Indian law — the Legal Metrology (Packaged Commodities) Rules, 2011 and the FSSAI labelling regulations. You photograph the label; we tell you what’s missing or wrong, and we cite the exact rule for every finding.

The key idea is a **division of labour**: the AI (a vision-LLM plus OCR) only *reads* the label — it extracts values. A separate **deterministic rule engine** *judges* compliance. The AI never decides pass or fail. That’s why every verdict is auditable, cites the law, and can never be a hallucination — and why updating to a new gazette notification is a JSON edit, not a retraining run.

Three follow-on sentences if they want more:

- **Rules are data, not code.** Each requirement lives in a JSON “rule pack” with its severity and its verbatim legal reference. Eight packs load automatically; adding a regulation is dropping in a file.
- **Two tiers.** Things a camera *can* verify (is the MRP printed? is it inclusive of all taxes? are the net-quantity numerals tall enough?) are scored. Things a camera *cannot* verify (is there really 80% milk-fat? are additive limits respected?) are surfaced as “verify in lab” but never scored — so the score is always defensible.
- **It degrades, never crashes.** Missing OCR, no API key, a photo that isn’t even a label — each has a defined, graceful outcome, not a 500 error.

1. The big picture

1.1 What problem it solves

Every retail package sold in India must legally declare a fixed set of things: who made it, the net quantity, the manufacturing date, the MRP “inclusive of all taxes”, consumer-care contact, and — for food — an FSSAI licence number, ingredients, nutrition, veg/non-veg mark, and more. Legal Metrology inspectors check this by hand, one pack at a time. It’s slow, subjective, and doesn’t scale to the volume of products on shelves and on e-commerce.

Label Jaano turns that manual check into a photograph. An inspector (or a consumer) photographs the label; the backend returns a structured verdict — **compliant**, **needs review**, or **non-compliant** — with a 0-100 score and an itemised list of exactly which legal declarations are missing or malformed, each tagged with the rule it breaks.

1.2 The one principle everything follows: *AI extracts, rules judge*

This is the single most important sentence in the whole project, and it is worth being able to explain three different ways.

Plain version. The AI’s only job is to read the label and tell us what it says. A separate, ordinary program then compares what the label says against the law and decides compliance. The AI is a *reader*; the rule engine is the *judge*.

Why it matters version. Large language models hallucinate. If you let an LLM decide “this label is compliant”, you can never fully trust or audit that verdict, and you can’t explain *why* in court. By confining the LLM to extraction — “the MRP text is ‘Maximum Retail Price Rs 45 inclusive of all taxes’” — and letting deterministic code decide, every verdict becomes reproducible, explainable, and tied to a citation. Same input, same output, every time.

Architecture version. The two halves meet at a single, well-defined data structure we call the **scan-input contract** — a normalised JSON object describing what’s on the label. The pipeline (AI side) *produces* it; the engine (rules side) *consumes* it. Neither knows about the other’s internals.

You can swap Gemini for another model, or PaddleOCR for another OCR, and the engine doesn't change a line. You can rewrite the engine and the pipeline doesn't care.

1.3 The three layers

The backend is three layers with a contract between the first two:

1. **The extraction pipeline** (backend/pipeline/) — photos in, scan-input JSON out. It runs OCR (for pixel geometry), a vision-LLM (for structured reading), and a pixel-to-millimetre calibration (for the font-size rule), then *fuses* them into one normalised object.
2. **The rule engine** (backend/rule_engine/) — scan-input JSON in, a ComplianceReport out. Pure Python standard library, zero dependencies. It selects the rule packs that apply to the product's category, runs every applicable check, and computes the verdict, score, and violations.
3. **The HTTP API** (backend/app/) — a thin FastAPI wrapper that exposes the engine and the pipeline over REST so the Flutter app, a web dashboard, and consumer mode can all call one service.

Data flows **pipeline** → **engine** → **API-response**. The scan-input contract is the seam between layers 1 and 2; that seam is why the project is testable, swappable, and honest.

1.4 The request lifecycle in one breath

A photo arrives at POST /scan/image. The pipeline OCRs it into word boxes, sends it to Gemini which returns structured fields plus a category, calibrates pixels to millimetres against a reference object in the frame, and fuses all of that into a scan-input dict. That dict is handed to the engine, which picks the Legal Metrology base pack plus any category packs (e.g. the FSSAI food pack), runs each declaration's checks, tallies weighted passes and fails into a score, derives a verdict from the worst violation severity, and returns a ComplianceReport. FastAPI serialises it to JSON. Total moving parts the *engine* depends on to do this: none but the standard library.

2. The tech stack — every choice and why

For each technology, know *what it is*, *what job it does here*, and — most important for a panel — *why it beats the obvious alternative*.

2.1 Python 3 (standard library) for the engine

What. The rule engine is written in plain Python using only the standard library — dataclasses, enum, re, datetime, json, pathlib. No third-party packages at all.

Why. The engine is the part that makes legal decisions, so it must be maximally trustworthy and portable. Zero dependencies means it runs anywhere with a bare python3, installs nothing, and can't break because a package upgraded. It's trivially unit-testable — the tests run with no pip install. When you tell a judge "the component that decides compliance has no external dependencies and is fully deterministic," that is a strong, concrete reliability claim.

Alternative rejected. Building the decision logic inside a web framework or an ML library would entangle the most sensitive code with heavy, changeable dependencies. We deliberately kept it pure.

2.2 Dataclasses (not Pydantic) inside the engine

What. Every engine model — Check, Declaration, Pack, ScanInput, ComplianceReport, etc. — is a @dataclass with an explicit from_dict builder.

Why. Dataclasses are standard library, so the engine stays dependency-free. Each model owns a `from_dict` that reads keys defensively with `.get(...)` and sensible defaults, so a slightly malformed pack or scan doesn't explode — it degrades. Pydantic is used only at the *API boundary* (Section 2.5), where request validation and auto-generated docs are worth a dependency. Inside the engine, we don't want that coupling.

2.3 Rules as JSON “rule packs” (not code, not a database, not a trained model)

What. Each regulation is a JSON file in `rulepacks/`. A pack lists declarations, each with an `id`, a human label, a verbatim `legal_reference`, a severity, a condition, and a list of checks. Eight packs ship today; the loader reads every `*.json` automatically.

Why. This is the design centrepiece, so the reasoning matters:

- **Auditable.** Anyone can open the JSON and read exactly what is enforced and under which clause. There's a `GET /rulepacks/{id}` endpoint that serves the raw pack for exactly this reason.
- **Updatable without code.** A gazette amendment becomes a JSON edit plus a `POST /reload`. No redeploy, no retraining, no engineer. That is the answer to “what happens when the law changes?”
- **Separation of concerns.** Domain experts curate rules; engineers maintain the engine. The two evolve independently.
- **Versionable.** Packs carry a `version` and `effective_date`, so you can point the engine at a different pack set (via an env var) for testing an amendment before it goes live.

Alternatives rejected (this is the teammates' question): a CSV is a strictly weaker JSON — it can't express nested checks, per-check messages, or conditions, and it's the same “data” idea with less power. Storing rules in a database and “retrieving by category” is exactly what the loader already does with files — a plain lookup, not “AI”. And *training or fine-tuning a model to decide compliance* throws away the whole advantage: you lose the citation, you lose auditability, verdicts become probabilistic and unexplainable, and you'd need a large hand-labelled dataset you don't have. Machine learning belongs on the *extraction* side (reading logos, classifying category), never on the *judging* side.

2.4 FastAPI + Uvicorn for the HTTP layer

What. `app/main.py` is a FastAPI application; Uvicorn is the ASGI server that runs it.

Why FastAPI. It gives three things for almost no code: (1) automatic request validation from type hints via Pydantic; (2) auto-generated interactive Swagger docs at `/docs`, which is a fantastic live demo — you can run a scan from the browser; (3) native `async` support, which we use so a slow OCR/Gemini call doesn't block the whole server (see `run_in_threadpool`, Section 4.6). It's modern, fast, and the standard choice for Python ML-backed APIs.

2.5 Pydantic v2 at the API boundary

What. `app/schemas.py` defines Pydantic models: `ScanRequest`, `ContextIn`, `FieldIn` for input; `ReportOut`, `ViolationOut`, `ReferenceStandardOut`, `PackInfo` for output.

Why. Pydantic validates and coerces incoming JSON *before* it reaches the engine, and it generates the OpenAPI schema that powers `/docs`. It's the right tool exactly at the edge, where untrusted input arrives. Note one subtlety we handle deliberately: FastAPI's `response_model=ReportOut` filters the response to declared fields, so `ReportOut` explicitly declares `reference_standards` — otherwise the two-tier data would be silently stripped from the response (this was a real bug we fixed).

2.6 Google Gemini 2.0 Flash for structured extraction

What. `pipeline/gemini.py` sends the label image(s) plus a rule-derived prompt to Gemini and parses a strict-JSON reply into structured fields, a category, detected symbols, and context flags.

Why a vision-LLM. Reading a real-world label is genuinely hard: text is curved, rotated, multilingual (Hindi + English), and laid out chaotically. A vision-LLM excels at “what does this text say and which declaration is it?” — turning a messy photo into “this is the MRP, this is the net quantity.”

Why Flash specifically: it’s fast and cheap, which matters for a phone-based, high-volume tool. Crucially, we call it with `temperature=0.0` and `response_mime_type: application/json` so the read is as deterministic and machine-parseable as the model allows.

The honest boundary. Gemini is unreliable at *precise pixel geometry* — it can’t tell you a letter is 1.4 mm tall. That’s why OCR exists alongside it.

2.7 PaddleOCR for word-level pixel geometry

What. `pipeline/ocr.py` wraps PaddleOCR to produce, for each recognised word, its text and a pixel bounding box (`x`, `y`, `w`, `h`).

Why OCR and an LLM. Rule 7(2)/Rule 8 of the Legal Metrology Rules mandates a *minimum letter height in millimetres*. To check that, you need real pixel heights of the actual glyphs — which is precisely what OCR bounding boxes give and the LLM does not. So the LLM says *what* the net quantity is; OCR says *how tall* its numerals are on the pack. They’re complementary, not redundant.

2.8 OpenCV for reference-based calibration

What. `pipeline/calibration.py` converts pixels to millimetres using a known-size reference in the frame — a credit/ID card (85.6 mm wide), an ArUco marker, or a manual scale. OpenCV is used (lazily) only for the ArUco-detection path.

Why. A photo has no inherent scale — the same label fills more pixels up close than far away. To turn “42 pixels tall” into “1.4 mm” you need something of known real size in the shot. A ubiquitous credit card is the pragmatic reference; ArUco markers are the precise option for a controlled setup.

2.9 pytest, plus bare-python3 test runners

What. `tests/` holds `test_engine.py`, `test_pipeline.py`, `test_api.py`, and `test_two_tier.py`. They run under `pytest`, and several also run as plain scripts (`python3 tests/test_engine.py`).

Why. The dual-mode design means the core logic can be verified with zero install — important for portability and CI. The engine and mock-pipeline tests need no dependencies; only the API tests need `httpx` (for FastAPI’s `TestClient`).

2.10 The mock layer — the unsung hero

What. Every heavy, external dependency (PaddleOCR, Gemini, OpenCV) has a deterministic, dependency-free **mock** path, toggled by `mock=True` or the `LABEL_JAANO MOCK=1` environment variable, and each layer *auto-detects*: if its library or API key is missing, it falls back to mock rather than crashing.

Why. This is what lets the entire photo→verdict path run offline, on any machine, in tests and demos, with no key and no GPU. It’s also how you demo reliably on stage without depending on conference Wi-Fi or a live API. Mock reads an optional `<image>.mock.json` sidecar so you can craft exact fixtures.

3. Architecture overview

3.1 The directory map

```

backend/
├── rule_engine/           # THE ENGINE – pure stdlib, zero deps, makes the decisions
│   ├── models.py         # dataclasses: rule packs, scan input, results
│   ├── loader.py         # read rulepacks/*.json, select + merge by category
│   ├── checks.py         # validators, symbols, conditions, per-check evaluation
│   ├── engine.py         # evaluate(scan, ruleset) -> ComplianceReport
│   └── __init__.py       # public API surface
├── pipeline/             # THE READER – photo -> scan-input (OCR + Gemini + calibration + fusion)
│   ├── types.py          # pure-stdlib dataclasses shared across the pipeline
│   ├── ocr.py            # PaddleOCR word boxes (real) / deterministic mock
│   ├── gemini.py         # Gemini 2.0 Flash structured read (real) / mock
│   ├── prompts.py        # rule-driven extraction prompt (never drifts from packs)
│   ├── calibration.py    # pixel->mm scale (card / ArUco / manual) for Rule 8
│   ├── fusion.py         # merge the three signals into the scan-input contract
│   ├── pipeline.py       # orchestrator: extract_scan_input / extract_and_evaluate
│   └── __init__.py
├── app/                  # THE API – FastAPI wrapper (the only part that needs pip install)
│   ├── schemas.py        # pydantic request/response models -> Swagger docs
│   └── main.py           # endpoints: /scan /extract /scan/image /health /rulepacks /reload
├── samples/              # demo scan JSONs + rendered demo photos + .mock.json sidecars
├── tests/                # test_engine / test_pipeline / test_api / test_two_tier
├── run_scan.py           # CLI: scan-input JSON -> verdict (exit code 0/1/2)
├── extract.py            # CLI: photo(s) -> scan-input (and optionally a verdict)
└── requirements.txt      # engine needs nothing; API + pipeline extras listed here

rulepacks/               # THE RULES – 8 versioned JSON packs (a sibling of backend/)

```

3.2 The scan-input contract — the seam

Everything hinges on one shape of JSON. The pipeline produces it; the engine consumes it; POST /scan accepts it directly. fields is keyed by **declaration id** — the *same ids* the rule packs use — which is the trick that lets extracted values and rules line up automatically without any glue.

```

{
  "category": "packaged_food",           // vision-LLM classification -> selects packs
  "raw_text": "full label text ...",     // used by format checks with target=raw_text
  "fields": {
    "mrp": {
      "value": "Maximum Retail Price Rs 10 inclusive of all taxes",
      "panel": "principal_display_panel", // for placement checks
      "height_mm": 3.1,                  // for font_height checks (null if unmeasured)
      "confidence": 0.94
    }
    // ... one entry per declaration the pipeline could read
  },
  "symbols_detected": ["veg_nonveg_mark", "fssai_logo"], // from the vision layer
  "context": {
    "is_imported": false,
    "has_allergens": false,
    "pdp_area_cm2": 300                  // principal-display-panel area for Rule 8
  }
}

```

Why this design wins: because the contract is explicit and declaration-ids are shared, adding a new rule means adding a declaration to a pack *and* (if it's a new field) teaching the prompt to read it — nothing in the engine's evaluation loop changes.

3.3 The eight rule packs and how they stack

One pack is the **base** (scope: "base"), the rest are **category** packs (scope: "category"). The base always applies; category packs apply when the scanned product's category matches their `applies_when.category_in`. They *stack* — a packaged food gets the base pack plus every food-related pack.

Pack	Scope	Applies to	Decl.	Ref. std.
legal_metrology_2011	base	always	10	0
fssai_food_2020	category	packaged_food, food, beverage, packaged_water	9	0
fssai_packaging_labelling_2011	category	packaged_food, food, beverage, packaged_water	8	0
fssai_contaminants_2011	category	food, beverage, water, nutraceutical, ...	0	5
fssai_organic_2017	category	packaged_food, food, beverage	3	1
fssai_fortification_2018	category	packaged_food, food, beverage	3	1
fssai_nutraceutical_2016	category	nutraceutical, health_supplement, ...	4	2
fssai_alcoholic_2018	category	alcoholic_beverage, wine, beer, spirit, liquor	11	3

Two things to notice. First, `fssai_contaminants_2011` has **zero** scored declarations and **five** reference standards — it's a pure Tier-2 pack (heavy-metal caps, etc. that a photo can't verify). Second, several food packs list overlapping categories, so a plain biscuit legitimately gets base + food_2020 + packaging_2011 + contaminants + organic + fortification stacked — most of whose narrow rules then sit dormant because their condition isn't met (see Section 4.3). This is why our tests assert pack membership with **subset** checks, never exact-set equality.

3.4 Two tiers: scored vs reference-only

- **Tier 1 — scored checks.** Photo-verifiable requirements. They become `CheckResults` and, on failure, `Violations`. They move the score and the verdict. Example: "is the MRP present and inclusive of all taxes, on the front panel?"
- **Tier 2 — reference standards.** Requirements that genuinely apply but a camera cannot confirm: composition minima, additive INS use-limits, heavy-metal caps, lab-only safety parameters. They ride along in the report as "applicable standard; verify in lab" and are **structurally incapable** of affecting the score, because they live on a separate data path (`ReferenceStandard`, never `Check`). Section 6 explains why that separation makes a scoring leak impossible rather than merely unlikely.

4. Deep code walkthrough

This section walks the real code. For each file: its job, its key functions, and the non-obvious decisions. Have the file open beside you.

4.1 rule_engine/models.py — the vocabulary of the whole system

This file defines every data structure the engine touches, in three families: **rule-pack models** (what the law requires), **scan models** (what a label actually says), and **result models** (what the engine produces). It's pure dataclasses so the engine stays dependency-free.

The enums

Severity(str, Enum) has CRITICAL, MAJOR, MINOR and a .weight property returning 3, 2, 1. Subclassing str means the enum serialises straight to JSON as its string value. The weight drives scoring — a failed critical check costs three times a failed minor one.

Outcome(str, Enum) is PASS, FAIL, SKIP. The distinction that matters: PASS and FAIL both *count* toward the score; SKIP is excluded entirely. SKIP means “not applicable, or couldn't evaluate” — e.g. a font-height check when no glyph height was measured. This is how the engine avoids penalising a label for something it simply couldn't see.

Verdict(str, Enum) is COMPLIANT, NEEDS_REVIEW, NON_COMPLIANT, and NO_LABEL (“no_label_detected”). That last one is the guard for “this image isn't even a product label.”

Rule-pack models

Check is a single test inside a declaration. It has a type (“presence”, “format”, “value”, “placement”, “font_height”, “symbol”) and then a superset of optional fields used by whichever type it is: regex/target/negate for format checks, validator/params for value checks, panel for placement, source/min_height_mm for font-height, symbol for symbol checks. Check.from_dict reads them all defensively with defaults (target defaults to “raw_text”, negate to False). One elegant detail: negate=True turns a format check into a **prohibition** — it passes when the pattern is *absent* — which is how “must NOT contain X” rules are expressed.

Declaration is one legal requirement: id, label, legal_reference, severity, required (default True), condition (default “always”), and a list of checks. The id is load-bearing — it's the key that links a rule to the extracted field of the same name.

ReferenceStandard is the Tier-2 model: id, label, legal_reference, authority, note, condition. Read its docstring in the code — it explicitly states these are “surfaced ... and NEVER turned into a Check / CheckResult / Violation.” It has a to_dict (it goes into the report) but note there is deliberately **no** path from a ReferenceStandard to a Check. That's the structural guarantee.

Pack bundles a pack's metadata (pack_id, label, authority, scope, version, applies_when), its declarations, an optional font_height_table, its scoring dict, and its reference_standards. The important method is:

```
def applies_to(self, category: str) -> bool:
    aw = self.applies_when or {}
    if aw.get("always"):
        return True
    return category in (aw.get("category_in") or [])
```

So the base pack (applies_when: {"always": true}) matches everything, and a category pack matches when the product's category is in its list. This one method is the entire pack-selection rule.

RuleSet is the *merged* result of all packs that apply to one scan: a flat list of declarations, the chosen font_height_table, the list of packs_applied, the scoring weights, and the merged reference_standards. The engine only ever sees a RuleSet, never individual packs.

Scan models (the normalised pipeline output)

Field is one extracted declaration value: value, raw_text, panel, height_mm, confidence. Its .present property is the definition of “did we read this?”: value is not None and str(value).strip() != "". And from_dict accepts a bare string as shorthand for {"value": "..."}, which keeps hand-written fixtures terse.

ScanContext is a big set of boolean flags plus pdp_area_cm2. The generic flags (is_imported, has_allergens, dimension_relevant, ...) gate base-pack rules; the narrow ones (is_organic, is_fortified, is_wine, has_aspartame, is_pan_masala, ...) gate category-pack rules. **Every flag defaults False.** That default is a safety mechanism, explained in Section 4.3: a narrow rule only fires when the pipeline *positively* detects the trait.

ScanInput ties it together: category, raw_text, fields (dict of Field keyed by declaration id), symbols_detected, and context. Its from_dict is the single entry point that parses a raw scan-input dict into typed objects.

Result models

CheckResult records the outcome of one check: which declaration, its legal reference and severity, the check type, the Outcome, the human message (shown on failure), and an engine-generated detail (why it passed or failed). Violation is the same shape but only created for failures. Both have to_dict for JSON.

ComplianceReport is the final product: verdict, score, category, packs_applied, violations, results (every check, not just failures), a summary dict, and reference_standards. Its to_dict is what the API serialises. Keeping *all* results (including passes and skips) — not just violations — is deliberate: the UI can show “here’s everything we checked,” which is far more convincing to an inspector than a bare verdict.

4.2 rule_engine/loader.py — loading and merging packs

This file answers two questions: *where are the rules?* and *which of them apply to this product?*

rulepacks_dir(override) resolves the rulepacks directory with a clear precedence: an explicit argument wins, then the LABEL_JAANO_RULEPACKS environment variable, then the default <project_root>/rulepacks. The default is computed as Path(__file__).resolve().parents[2] / "rulepacks" — i.e. up two levels from backend/rule_engine/loader.py to the project root. The env-var override is what lets you point the engine at an amended pack set for testing without touching code.

load_packs(...) globs *.json in that directory (sorted, so order is deterministic), parses each into a Pack via Pack.from_dict, and raises a clear FileNotFoundError if the directory is missing or empty. Sorting matters because it makes “which pack defined the font table first” reproducible.

load_pack_dicts(...) does the same but keeps each pack as the *raw* parsed dict, keyed by pack_id. This exists purely so GET /rulepacks/{id} can serve the unmodified rule text — every check and regex — for auditing. It’s a separate function precisely so the audit endpoint shows exactly what’s on disk, not a re-serialised approximation.

build_ruleset(category, packs=None, ...) is the important one — the merge algorithm:

```
applicable = [p for p in packs if p.applies_to(category)]
# base packs first so category packs can override by id
applicable.sort(key=lambda p: 0 if p.scope == "base" else 1)
```

```

merged: dict[str, Declaration] = {}  # declaration id -> declaration (later wins)
order: list[str] = []
font_table = None
weights = None
ref_merged: dict = {}
ref_order: list[str] = []

for pack in applicable:
    if font_table is None and pack.font_height_table:
        font_table = pack.font_height_table
    if weights is None and pack.scoring.get("weights"):
        weights = pack.scoring["weights"]
    for decl in pack.declarations:
        if decl.id not in merged:
            order.append(decl.id)
            merged[decl.id] = decl  # later pack with same id overrides
    for ref in pack.reference_standards:
        if ref.id not in ref_merged:
            ref_order.append(ref.id)
            ref_merged[ref.id] = ref

```

Five decisions are encoded here, and each has a reason:

1. **Filter by category** using `applies_to` — only relevant packs are considered.
2. **Sort base-first** so that when a category pack reuses a base declaration id, the category version *overrides* it (because it's processed later and "later wins" in the dict). This lets, say, the FSSAI pack tighten a base rule for food without duplicating it.
3. **First font table wins.** The base Legal Metrology pack owns the Rule 7(2) font-height table; taking the first one found means the base table is authoritative.
4. **First weights win**, defaulting to {critical:3, major:2, minor:1} if no pack specifies them.
5. **Merge reference standards with de-dup by id**, preserving first-seen order — so overlapping food packs don't list the same heavy-metal cap twice.

The function returns a single RuleSet. Note `order/ref_order` lists preserve a stable, readable ordering in the output rather than relying on dict insertion quirks.

4.3 rule_engine/checks.py — validators, symbols, conditions, and the dispatcher

This is what the code calls the *engine contract*: when a rule changes you edit JSON, but when you need a genuinely new *kind* of check, you add it here, in one of the registries.

Value validators

Each validator has the uniform signature `fn(value, scan, params) -> (bool, detail)`. Passing the whole scan in (not just the value) is what lets cross-field validators work — e.g. checking expiry is after manufacture needs to read *another* field. The registry `VALIDATORS` maps a name to a function; a pack's value check just names one. The current set:

- `nonempty_address` — requires ≥ 10 chars and either a 6-digit PIN or a comma-plus-length heuristic. Catches "Pune" as too thin to be a full address.
- `standard_metric_unit` — regex for g/kg/mg/l/ml/... or a count, enforcing metric declarations.
- `positive_currency` — parses an amount (tolerating Rs/INR/currency symbol and thousands separators) and requires it to be > 0 .
- `valid_month_year_not_future` — parses dates and rejects a manufacture date in the future.

- `valid_expiry_after_mfg` — parses expiry, then compares against the separately-extracted `manufacture_date` field if present, else against a second date inside the same value; a lone best-before date is accepted.
- `phone_or_email_present` — regex for an email or an Indian phone / 1800 toll-free number.
- `fssai_license_14digit` — requires a 14-digit number (the FSSAI licence format).

`_parse_dates(text)` is the shared date-wrangler behind the date validators. It's deliberately forgiving — it handles `dd/mm/yyyy`, `mm/yyyy`, `mm-yy`, `Mon YYYY`, `Mon-YY` — because label dates are printed a dozen ways. `_norm_year` maps a two-digit year into the 2000s and clamps to a sane 1990–2100 window. The philosophy: extract every plausible date, then reason over them, rather than demanding one rigid format.

Symbols

`KNOWN_SYMBOLS` is a set of symbol names the engine understands (`fssai_logo`, `veg_nonveg_mark`, `india_organic`, `fortification_plus_f_logo`, ...). `symbol_detected(symbol, scan)` just checks membership in `scan.symbols_detected`. For the MVP the CV layer isn't wired in, so symbols come from what the vision-LLM reports; the comment notes you can later swap in a real detector keyed by the same names *with no pack changes* — the registry is the stable interface.

Condition tokens — and the fail-safe

`condition_met(condition, scan)` maps a condition string to a boolean by reading `scan.context`:

```
table = {
    "always": True,
    "imported": ctx.is_imported,
    "has_allergens": ctx.has_allergens,
    "is_organic": ctx.is_organic,
    "is_wine": ctx.is_wine,
    "has_aspartame": ctx.has_aspartame,
    "is_pan_masala": ctx.is_pan_masala,
    # ... one entry per context flag, plus compound triggers ...
    "low_alcohol_non_wine": ctx.is_low_alcohol and not ctx.is_wine,
}
return table.get(condition, True) # unknown token -> True (fail-safe)
```

Two things to be able to defend:

1. **The default is True, not False.** An *unregistered* condition token evaluates to True — meaning “check it anyway.” This is a fail-safe: if a pack references a condition the engine doesn't know, we'd rather over-check (and possibly SKIP for lack of data) than silently ignore a legal requirement.
2. **Because unknown → True, every context flag a pack relies on must be a registered flag defaulting False.** That's the mirror-image invariant. A narrow rule (say, an aspartame warning) is gated on `has_aspartame`, which defaults False, so it stays dormant unless the pipeline positively sets it. Register the flag in three places kept in sync: `ScanContext` (engine), `CONTEXT_KEYS` (prompt), and `ContextIn` (API schema). Miss one and the flag gets dropped before it reaches the engine.

Font-height table lookup

`lookup_min_height(table, area_cm2)` walks the threshold rows and returns the first `min_height_mm` whose `max_area_cm2` is None (the catch-all top band) or `>= area_cm2`. So a 300 cm² panel lands in the “≤500 cm²” band and requires 2.5 mm numerals. No area, no table → None (which makes the check SKIP).

The per-check dispatcher

`evaluate_check(check, decl, scan, ruleset) -> (Outcome, detail)` is the single switchboard every check flows through. The controlling idea in its docstring: **field-dependent checks SKIP when the field is absent, so we don't double-penalise.** If the net quantity wasn't extracted at all, its presence check already reports the failure; its value/placement/font_height checks then SKIP rather than piling on more failures for the same root cause. Per type:

- `presence` — PASS if the field is present; else FAIL if the declaration is required, SKIP if optional.
- `symbol` — PASS/FAIL on `symbol_detected`. This one always evaluates (it doesn't depend on a text field).
- `format` — matches `check.regex` against either `raw_text` or the field's normalized value (`target`). Honours negate (prohibition). A bad regex SKIPS with a detail rather than crashing. Field-targeted format on an absent field SKIPS.
- `value` — looks up the named validator and runs it; unknown validator → SKIP; absent field → SKIP.
- `placement` — compares the field's panel to the check's expected panel; unknown panel → SKIP.
- `font_height` — needs a measured `height_mm`; uses the check's own `min_height_mm` or looks one up from the ruleset's font table by PDP area; missing either → SKIP; else PASS/FAIL on the mm comparison.
- anything else → SKIP with "unknown check type."

Every branch returns a human-readable detail string, which is why the final report can explain *why* each check landed where it did.

4.4 rule_engine/engine.py — the judge

This is where a scan becomes a verdict. The public entry points are `evaluate(scan, ruleset)` and the convenience `evaluate_scan(scan)` which builds the ruleset for you (`build_ruleset(scan.category)` then `evaluate`).

The main loop

```
def evaluate(scan, ruleset):
    if not _looks_like_label(scan):
        return _no_label_report(scan, ruleset)    # guard, see below

    results, violations = [], []
    for decl in ruleset.declarations:
        if not condition_met(decl.condition, scan):
            continue                               # rule not applicable to this product
        field = scan.fields.get(decl.id)
        present = field.present if field else False
        if not decl.required and not present:
            continue                               # optional + absent -> ignore entirely
        for check in decl.checks:
            outcome, detail = evaluate_check(check, decl, scan, ruleset)
            results.append(CheckResult(...))
            if outcome is Outcome.FAIL:
                violations.append(Violation(...))

    verdict = _verdict(violations)
    score = _score(results, ruleset.weights)
    summary = _summary(results, violations)
    return ComplianceReport(...,
        ↪ reference_standards=_collect_reference_standards(ruleset, scan))
```

Read the two continues carefully, because they encode real legal nuance:

- **Condition gate.** A declaration whose condition isn't met is skipped *before any check runs* — it emits no `CheckResult` at all. A country-of-origin rule simply doesn't exist for a domestic product; it shouldn't show up as a pass, a fail, or even a skip.
- **Optional-and-absent gate.** An *optional* declaration that wasn't extracted is skipped entirely too. Optional declarations (e.g. an advisory best-before on a non-food) only count *when present* — printing one holds you to getting it right, but not printing one isn't a violation.

A required declaration that's absent does *not* hit these continues — it flows into its checks, where the presence check FAILs. That's the intended asymmetry between required and optional.

The “is this even a label?” guard

Before judging, `_looks_like_label(scan)` decides whether the image is plausibly a packaged-commodity label at all. Without this, photographing a book cover or a face would produce a confidently “non-compliant product” verdict, which is nonsense and erodes trust. The guard returns `True` if *any* of these hold:

- at least one field was extracted (`any(f.present ...)`), or
- any regulatory symbol was detected, or
- any meaningful context flag is set (`imported, has_allergens, ...`), or
- the raw text matches `_LABEL_SIGNAL_RE` — a regex for the unmistakable fingerprints of a label: “net qty/quantity/wt”, “MRP”, “maximum retail price”, “mfg/mfd/packed on/best before/use by/expiry”, “fssai”, “batch/lot no”, or a number followed by a weight/volume unit.

If none hold, `_no_label_report` returns a clean report with verdict `NO_LABEL`, score `0.0`, empty results/violations, and a summary note “No packaged-commodity label detected in the image.” It's a distinct, honest outcome — not a failure of the product.

Collecting reference standards

`_collect_reference_standards(ruleset, scan)` iterates the ruleset's `reference_standards`, keeps those whose condition is met (same `condition_met` gate as declarations), and returns their `to_dict()`s. The crucial line to point at when asked “how do you guarantee these don't affect the score?": they're gathered here, **after** results/violations are already computed, and they were never Checks to begin with — so there is no code path by which they enter `_score` or `_verdict`. The guarantee is structural, not a flag we remembered to check.

Verdict, score, summary

`_verdict(violations)` — pure severity logic: any `CRITICAL` violation → `NON_COMPLIANT`; else any `MAJOR` → `NEEDS_REVIEW`; else `COMPLIANT`. Minor-only failures never downgrade past compliant. This matches the packs' stated `verdict_rule`.

`_score(results, weights):`

```
passed = sum(w(r.severity) for r in results if r.outcome is Outcome.PASS)
scored = sum(w(r.severity) for r in results if r.outcome in (PASS, FAIL))
return round(100.0 * passed / scored, 1) if scored else 100.0
```

So the score is **weighted pass ratio**: the weighted sum of passed checks over the weighted sum of all *scored* (pass or fail) checks. SKIPs are excluded from both numerator and denominator — a check we couldn't evaluate neither helps nor hurts. If nothing was scored at all, the score is `100.0` (there was nothing to fail). Weights come from the ruleset (critical 3, major 2, minor 1 by default), so failing a critical declaration hurts the score far more than a minor one — the score and the verdict tell a consistent story.

`_summary(results, violations)` produces the counts the UI shows: totals of checks, passed, failed, skipped, total violations, and violations broken down by severity. It's derived data — everything

in it is recomputable from results and violations — kept in the report so clients don't have to recompute it.

4.5 rule_engine/__init__.py — the public surface

Small but intentional: it re-exports exactly the names a caller should use — `evaluate`, `evaluate_scan`, `build_ruleset`, `load_packs`, and the core models (`ScanInput`, `ScanContext`, `Field`, `RuleSet`, `ComplianceReport`, `Violation`, `Severity`, `Outcome`, `Verdict`) — via `__all__`, and sets `__version__`. This is the difference between “import from a package” and “reach into its internals”: callers do `from rule_engine import ScanInput, evaluate_scan` and never depend on file layout.

4.6 The pipeline — backend/pipeline/

The pipeline turns raw photos into the scan-input contract. Its guiding rule: **every heavy dependency is lazy-imported**, so the whole package imports and runs (in mock mode) on a machine with no PaddleOCR, no Gemini SDK, and no OpenCV. The imports happen *inside* the functions that actually need them.

types.py — intermediate data types

Pure-stdlib dataclasses for the stages between photo and contract:

- `OcrWord` — one recognised token with its pixel box (`x`, `y`, `w`, `h`) and confidence, plus computed properties `x/y/width/height` and centre `cx/cy`. `height` is the glyph height in pixels that calibration multiplies by mm-per-pixel.
- `OcrResult` — all words for one image, plus image dimensions and the engine name. Its `full_text` property joins the words into a reading-order-ish string used as part of the engine's `raw_text`.
- `GeminiField` — one declaration value as the LLM read it: `value`, a coarse panel (`front/back/side/unknown`), the `source_image` index it came from, and confidence. `from_dict` also accepts a bare string.
- `GeminiExtraction` — the full structured read of one product across images: `category`, `fields` (dict of `GeminiField`), `symbols_detected`, `context` (raw booleans), `raw_text`, `model`.
- `CalibrationResult` — the pixel→mm result: `mm_per_px`, the method used (`aruco/card/manual/none`), confidence, and a human detail. `.calibrated` is `True` only when `mm_per_px` is a positive number.

ocr.py — PaddleOCR wrapper (real + mock + graceful degradation)

`run_ocr(image, mock=None, lang="en")` is the entry point. If `mock` is `None` it auto-detects via `LABEL_JAANO_MOCK`. The real path (`_paddle_ocr`) lazily imports PaddleOCR (cached in a module-level singleton, because building the model is expensive), decodes the image with `PIL/numpy`, runs OCR, and converts PaddleOCR's 4-point polygons into axis-aligned (`x`, `y`, `w`, `h`) boxes.

The robustness story here is worth telling in full, because it's a common judge question (“what if OCR isn't installed / fails?”):

```
try:
    return _paddle_ocr(image, lang=lang)
except (ImportError, RuntimeError):
    # extras not installed -> no word boxes, engine=unavailable
    return OcrResult(words=[], engine="unavailable")
except Exception:
    # extras present but THIS image failed -> no word boxes, engine=error
    return OcrResult(words=[], engine="error")
```

OCR's *only* job is supplying pixel geometry for the Rule 8 font-height check. So if it's missing or fails, we return zero word boxes and let the pipeline continue on Gemini alone — the font-height check

simply SKIPs downstream (no measured height), and every other check proceeds. **A broken OCR degrades one check; it never 500s the scan.** (This exact fix is why the app stopped erroring when numpy wasn't present.)

The mock path (`_mock_ocr`) reads an `<image>.mock.json` sidecar's `ocr_words` if present, else returns a small synthetic set of measurable word boxes (a net-quantity line and an MRP line) so calibration/font-height have something to chew on in demos.

gemini.py — vision-LLM structured extraction (real + mock)

`extract_fields(images, declaration_ids, symbols, categories, mock=None, model=...)` is the entry point. Auto-detect for mock here is smart: it mocks if `LABEL_JAANO MOCK` is set **or if no API key is configured** — so a missing key falls back to a working offline read instead of crashing.

The real path (`_gemini_extract`) lazily imports `google.generativeai`, reads the key from `GEMINI_API_KEY/GOOGLE_API_KEY`, builds the parts list `[prompt, image, image, ...]`, and calls `generate_content` with `temperature=0.0` and `response_mime_type="application/json"` for a deterministic, JSON-only reply. `_parse_json` is defensively written — it strips ````json` fences if the model adds them, and as a last resort grabs the outermost `{...}` — because models occasionally wrap JSON in prose despite instructions. If it truly can't parse, it raises a clear error.

The mock path (`_mock_extract`) uses a sidecar's extraction block if present, else returns a canned, realistic compliant packaged-food read (a full set of fields, `veg_nonveg_mark` + `fssai_logo`, a plausible `raw_text`). This canned data is what makes the offline demo produce a real, non-trivial report.

prompts.py — the rule-driven prompt

`build_extraction_prompt(declaration_ids, symbols, categories)` returns the instruction string sent to Gemini. The defining feature: **the prompt is generated from the live rule packs**, not hard-coded. The orchestrator passes in the declaration ids and symbol names the *currently loaded* packs define, so the extractor asks for exactly what the current regulations require. Add a pack, and the extractor automatically starts asking for its fields — the extractor and the rules can never drift apart. The prompt instructs Gemini to return a single JSON object with `category`, `fields` (keyed by declaration id, verbatim transcription, per-field panel/source/confidence), `symbols_detected`, a context object of booleans, and a full `raw_text`. `CONTEXT_KEYS` lists every context flag — and the module comment warns it must stay in sync with `ScanContext` and `ContextIn`, for the reason in Section 4.3.

calibration.py — the pixel→millimetre bridge for Rule 8

`estimate_scale(reference, image)` computes `mm_per_px` from a reference descriptor: manual (operator gives the scale directly), card (`width_mm / bbox_width_px`, defaulting to the 85.6 mm credit-card standard), or aruco (detects a marker with OpenCV and divides its known side length by its measured pixel side). Only the ArUco path needs OpenCV, and it's lazy — card/manual and all the measurement math are pure stdlib, so they unit-test anywhere. Every failure mode returns a `CalibrationResult` with `method/detail` explaining what went wrong, never an exception.

`measure_height_mm(field_value, ocr_words, mm_per_px)` is the clever bit. It finds the OCR words that make up a declaration's value (token overlap), takes the **median** of their pixel heights — median, so one stray tall/short box doesn't skew it — and scales to mm. It prefers multi-character tokens, falling back to single-char ones (so a lone unit like "g" can still be measured). No scale or no matching words → `None`, and the font-height check SKIPs rather than guessing.

`pdp_area_cm2(pdp_bbox_px, mm_per_px)` converts the front-panel pixel box to `cm2` for the Rule 8 table lookup (`w_mm * h_mm / 100`).

fusion.py — merging the three signals into the contract

`fuse(extraction, ocr_results, calibrations, reference, context_overrides)` is where Gemini + OCR + calibration become one scan-input dict. What it does, and the one thing that matters most:

- **Fields.** For each Gemini field with a value, it emits `{value, panel?, confidence?, height_mm?}`. The `height_mm` is computed by calling `measure_height_mm` against the OCR words *of the image that field came from* (`gf.source_image`, clamped to a valid index).
- **The critical mapping.** The engine's placement check compares `field.panel` to the pack's exact string (`"principal_display_panel"`). The LLM only gives coarse `"front"/"back"`. So `_PANEL_MAP` translates `front` → `principal_display_panel`, `back` → `back_panel`, etc. Without this translation, a correctly-placed net quantity would be scored as *misplaced*. This is the kind of subtle seam that shows the design is thought-through.
- **raw_text.** Union of the OCR transcription and the LLM transcription, for maximum recall on text-pattern checks.
- **Symbols.** Filtered to only names in `KNOWN_SYMBOLS` — the engine ignores symbols it doesn't understand, so fusion drops them early.
- **Context.** Keeps only booleans in `CONTEXT_KEYS`, adds the calibrated `pdp_area_cm2`, and lets an officer's `context_overrides` win last. Officer-in-the-loop by design.

pipeline.py — the orchestrator

`extract_scan_input(images, reference, context_overrides, mock, category_hint)` runs the four steps in order: OCR every image, one Gemini call over all images, calibrate the reference image once and broadcast that scale to every panel, then `fuse`. `_rule_vocabulary()` pulls the declaration ids, symbols, and categories from the live packs to feed the prompt. `category_hint` lets a caller force the category. `extract_and_evaluate(...)` is the convenience wrapper that then runs the engine and returns `(scan_input_dict, ComplianceReport)`. The `mock` flag threads through every layer, so `mock=True` gives a fully offline photo→verdict path and `mock=None` lets each layer independently light up its real model when available.

4.7 The API — backend/app/

schemas.py — the Pydantic boundary

These models exist so FastAPI can validate input and generate the Swagger docs; the engine itself stays pure-dataclass. Input side: `FieldIn` (`value/raw_text/panel/height_mm/confidence`), `ContextIn` (every context flag, mirroring `ScanContext`), and `ScanRequest` (`category, raw_text, fields, symbols_detected, context`) — which carries a full worked examples block so `/docs` shows a ready-to-run request. Output side: `ViolationOut`, `CheckResultOut`, `ReferenceStandardOut`, `ReportOut`, and `PackInfo`.

Two deliberate details to be able to explain:

1. **ContextIn mirrors ScanContext exactly** — the same 19 flags — and the comment says why: every flag a pack gates on must appear here, or the request model silently drops it and the rule stops firing. It's one of the three places kept in sync.
2. **ReportOut explicitly declares reference_standards.** Because FastAPI's `response_model` filters output to declared fields, omitting this field would strip the entire Tier-2 section from the JSON response — which was a real bug. Declaring `ReferenceStandardOut` is what keeps the two-tier data flowing to clients.

main.py — the FastAPI app and endpoints

The top of the file does one important thing before the imports: it puts `backend/` on `sys.path` so the sibling `rule_engine` and `pipeline` packages import cleanly whether you launch with `uvicorn app.main:app` or as a module. Then it constructs the FastAPI app and adds permissive CORS (`allow_origins=["*"]`) for local dev — with a comment to tighten it before production, since the dashboard, Flutter app, and consumer mode all live on other origins.

Pack caching. `get_packs()` and `get_pack_dicts()` are wrapped in `@lru_cache(maxsize=1)`, so packs are parsed once and reused across requests — and `POST /reload` clears those caches to hot-apply a gazette edit. That’s the mechanism behind “update the law without a restart.”

The endpoints:

- `GET /health` — returns status, version, and how many packs loaded (with their ids). Returns 503 if the packs won’t load, so a broken deploy fails loudly at the health check.
- `GET /rulepacks` — a `PackInfo` summary of every loaded pack.
- `GET /rulepacks/{id}` — the full raw JSON of one pack (the audit endpoint), 404 with the known ids if the id is unknown.
- `POST /scan` — the core JSON endpoint. Converts the `ScanRequest` to `ScanInput`, builds the ruleset from `scan.category` using the cached packs, evaluates, and returns `report.to_dict()`. This is the deterministic path with no AI in it at all.
- `POST /extract` — multipart photo upload → the normalised scan-input JSON (runs the pipeline, no scoring). Lets a client inspect or correct the extraction before scoring.
- `POST /scan/image` — multipart photo upload → full `ComplianceReport` in one shot (extract + judge).
- `POST /reload` — clears the pack caches and eagerly reloads (so a bad pack surfaces its error here, not on the next scan).

Async and blocking work. The image endpoints are async def, but OCR and Gemini are blocking, CPU/IO-heavy calls. They’re dispatched with `await run_in_threadpool(...)` so a slow scan doesn’t block the event loop and freeze the server for other requests. Input helpers (`_read_images`, `_parse_json_form`, `_parse_mock_form`) validate uploads and parse the optional reference/context/mock form fields, returning clean 400s on bad input and 503s when an optional dependency or API key is missing. Nothing crashes the worker.

4.8 The command-line runners

These aren’t just demos — they’re how you verify the system without the app, and they return CI-friendly exit codes (0 compliant / 1 needs-review / 2 non-compliant).

`run_scan.py` takes a scan-input JSON file, runs `evaluate_scan`, and pretty-prints the verdict, score, per-severity violation counts, each violation with its legal reference and message, and — importantly — a “NOT EVALUATED” section listing everything that SKIPped, so nothing looks silently ignored. `--json` emits the raw report. Its `render(report)` function is reused by `extract.py`.

`extract.py` takes one or more image paths, runs `extract_scan_input` (or, with `--evaluate`, `extract_and_evaluate`), and prints the scan-input JSON or the rendered report. It supports `--reference/--context` as inline JSON or @file, `--category` to force the class, and `--mock` for the offline path. Together the two CLIs cover both halves: `extract.py` exercises the reader, `run_scan.py` exercises the judge, and `extract.py --evaluate` exercises both end to end.

5. End-to-end trace: one photo to a verdict

Follow a single scan through every function. This is the story to tell when a judge says “walk me through what happens when I take a photo.”

Input. An inspector opens the Flutter app and photographs a biscuit packet’s front and back panels, with a credit card laid in the front shot as a size reference. The app POSTs both images (plus a reference describing the card) to `POST /scan/image`.

1. **API entry** (`app/main.py::scan_image`). FastAPI receives the multipart upload. `_read_images` reads the two files into bytes (rejecting empties with a 400). `reference` and `context` form fields are parsed from JSON. The work is dispatched via `run_in_threadpool(extract_and_evaluate, imgs, ...)` so the event loop stays free.

2. **Orchestration** (`pipeline/pipeline.py::extract_and_evaluate → extract_scan_input`). `_rule_vocabulary()` loads the live packs and collects the declaration ids (`mrp`, `net_quantity`, `fssai_license`, ...), known symbols, and categories to ask for.
3. **OCR** (`pipeline/ocr.py::run_ocr`, once per image). PaddleOCR returns word boxes: `("500", box)`, `("g", box)`, `("MRP", box)`, etc., each with a pixel height. If PaddleOCR isn't installed, this returns empty words with `engine="unavailable"` and the scan continues — only the font-height check will later SKIP.
4. **Vision-LLM** (`pipeline/gemini.py::extract_fields`). One Gemini call over both images with the rule-driven prompt. Gemini returns strict JSON: `category: "packaged_food"`, `fields (mrp = "Maximum Retail Price Rs 45 inclusive of all taxes" on the front, net_quantity = "Net Qty 500 g" on the front, manufacturer_details, fssai_license "...10012345678901", ...)`, `symbols_detected: ["veg_nonveg_mark", "fssai_logo"]`, context booleans, and a full `raw_text`. `_parse_json` tolerates any stray fences.
5. **Calibration** (`pipeline/calibration.py::estimate_scale`). The card reference gives `mm_per_px = 85.6 / card_width_px`. That single scale is broadcast to both panel images.
6. **Fusion** (`pipeline/fusion.py::fuse`). For each field: keep the value, map the LLM's "front" to "principal_display_panel", and measure glyph height — e.g. `measure_height_mm("Net Qty 500 g", front_ocr_words, mm_per_px)` takes the median pixel height of the matching words and scales it to, say, 2.6 mm. `raw_text` is the union of OCR and LLM text; symbols are filtered to known names; `pdp_area_cm2` is computed from the front-panel box; officer overrides (none here) would win last. Out comes the scan-input dict.
7. **Parse to typed input** (`rule_engine/models.py::ScanInput.from_dict`). The dict becomes typed `Field/ScanContext/ScanInput` objects.
8. **Build the ruleset** (`rule_engine/loader.py::build_ruleset("packaged_food")`). `applies_to` selects the base Legal Metrology pack plus the food-category packs; they're sorted base-first and merged into one `RuleSet` (declarations de-duplicated by id, base font table chosen, reference standards merged).
9. **The label guard** (`engine.py::_looks_like_label`). Fields were extracted, so it's clearly a label — proceed. (Had this been a photo of a book, we'd short-circuit to a `NO_LABEL` report here.)
10. **Evaluate** (`engine.py::evaluate`). For each applicable declaration, `condition_met` decides if it applies (country-of-origin is skipped for this domestic product; the aspartame rule stays dormant because `has_aspartame` is `False`). For each check, `evaluate_check` runs the right branch: `mrp`'s presence `PASS`es, its two format regexes match "maximum retail price" and "inclusive ... all taxes" (`PASS`), `positive_currency` finds 45 (`PASS`), `placement` confirms the front panel (`PASS`); `net_quantity`'s `font_height` compares 2.6 mm against the Rule 7(2) minimum for the panel area (`PASS` or `FAIL` with a precise detail). Each check appends a `CheckResult`; each failure also appends a `Violation`.
11. **Verdict + score + summary** (`engine.py`). `_verdict` inspects violation severities; `_score` computes the weighted pass ratio (skips excluded); `_summary` tallies counts. `_collect_reference_standards` gathers any applicable Tier-2 items (e.g. contaminant caps) as "verify in lab" — after scoring, on a separate path, so they can't move the number.
12. **Serialise** (`app/main.py`). `report.to_dict()` is returned; FastAPI validates it against `ReportOut` (which includes `reference_standards`) and sends JSON to the app, which renders the verdict banner, the score, the violations each citing their rule, the skipped items, and the lab-only reference standards.

The whole judging half — steps 7 through 11 — used nothing but the Python standard library, and given the same scan-input would return byte-identical results every time.

6. The two-tier model, in depth

This is a differentiator judges love, because it shows legal maturity — you understood that not everything on a label can be checked from a photo, and you designed for that honestly instead of pretending.

6.1 The problem

A label makes two kinds of legal claims. Some are **verifiable from the image**: is the MRP printed, is it inclusive of all taxes, are the net-quantity numerals tall enough, is the FSSAI number 14 digits. Others are **real legal requirements a camera fundamentally cannot confirm**: does butter actually contain $\geq 80\%$ milk fat, are additive INS use-limits respected, are heavy-metal contaminants under their caps, does the product pass a microbiological safety limit. Those need a laboratory, not a photograph.

A naïve compliance tool ignores the second kind (looks thorough but isn't) or, worse, scores them as pass/fail based on nothing (produces confident nonsense). We do neither.

6.2 The design

Tier-2 provisions are modelled as `ReferenceStandard`, a completely separate dataclass from `Check`. They flow through their own path:

- They're loaded into the pack's `reference_standards` list (not declarations).
- `build_ruleset` merges them into `RuleSet.reference_standards` (de-duped by id).
- `evaluate` gathers them via `_collect_reference_standards`, which runs *after* scoring and only applies the `condition_met` gate — it never constructs a `Check`, a `CheckResult`, or a `Violation`.
- They surface in `ComplianceReport.reference_standards` and in the API's `ReportOut.reference_standards`, labelled “applicable standard; verify in lab.”

6.3 Why the guarantee is *structural*, not a promise

The strong claim — and it's true — is that a reference standard **cannot** move the score or verdict, no matter what. Not “we're careful not to let it,” but “there is no code path.” Here's the argument you can make out loud:

- The score comes only from `CheckResults` with outcome `PASS` or `FAIL` (`_score`).
- The verdict comes only from `Violation` severities (`_verdict`).
- `CheckResults` and `Violations` are created in exactly one place: the check loop in `evaluate`, iterating `ruleset.declarations` and their `Checks`.
- `ReferenceStandards` are never in declarations and never become `Checks` — there is no constructor, no branch, no conversion from a `ReferenceStandard` to a `Check` anywhere in the code-base.

So a scoring leak isn't merely unlikely; it's unrepresentable in the data model. This is enforced by `tests/test_two_tier.py`, which includes a test that recomputes the score from results and asserts reference standards never change it. If someone later tried to wire the two together, that test would fail.

6.4 How to say it in one line

“We split requirements into what a photo can verify and what needs a lab. The photo-verifiable ones are scored and cite their rule; the lab-only ones are surfaced as ‘verify in lab’ but live on a separate data path that literally cannot touch the score. So our score is always defensible — it only reflects things we could actually see.”

7. Testing and verification

Know this section cold — “how do you know it works?” is a guaranteed question.

7.1 The test suites

- `tests/test_engine.py` — end-to-end and unit tests for the engine, no dependencies. It scores the compliant sample at 100 and the deliberately non-compliant sample lower, checks the verdict logic, and exercises individual validators.
- `tests/test_pipeline.py` — the pipeline in mock mode: OCR mock, Gemini mock, calibration math, fusion’s panel-mapping and height measurement — all dependency-free.
- `tests/test_two_tier.py` — the two-tier guarantees: reference standards appear when their condition is met, and (the key test) recomputing the score from results proves reference standards never change it.
- `tests/test_api.py` — the HTTP layer via FastAPI’s `TestClient` (needs `httpx`, runs in-process, no live server).

They run two ways: under `pytest`, or several as bare scripts (`python3 tests/test_engine.py`) with zero install — which is how the core logic is verified even in a locked-down environment.

7.2 The subset-assertion lesson (a real gotcha we fixed)

When the six FSSAI packs were added, four tests broke — not because the engine was wrong, but because they hard-coded the exact set of applied packs (`set(packs_applied) == {"legal_metrology_2011", "fssai_food_2020"}`). Several new packs legitimately list `packaged_food/beverage` in their `category_in`, so they now stack onto every food scan. The fix was to assert **subset** membership (`{"legal_metrology_2011", "fssai_food_2020"} <= set(packs_applied)`), not equality. The lesson, now a standing rule: because category packs stack, tests must assert “*at least these packs applied*,” never “*exactly these*.” Category packs whose declarations are all condition-gated on narrow flags add no scored results to an unflagged scan — the engine continues past a gated-out declaration — so only pack *membership* grows, not the score.

7.3 The verification philosophy

Two habits worth stating: first, the deterministic engine means tests are exact — a scan-input maps to one report, so assertions are precise, not fuzzy. Second, the mock layer means the full photo→verdict path is testable offline, so “does the whole thing work end to end?” is answerable in CI without a GPU or an API key. The CLIs’ exit codes (0/1/2) also make the engine scriptable into a CI gate.

8. Extending, scaling, and the honest roadmap

8.1 How you extend it (three cases, increasing effort)

- **A rule changed / a new gazette notification.** Edit the JSON pack, `POST /reload`. No code, no restart.
- **A whole new regulation** (cosmetics, drugs, BIS electronics). Drop in a new pack file with an `applies_when.category_in`; it’s auto-selected. The extractor starts asking for its fields automatically because the prompt is built from the packs.
- **A genuinely new kind of check** (a new validator or symbol). Add a function to the `VALIDATORS` registry or a name to `KNOWN_SYMBOLS` in `checks.py`, then reference it from a pack. This is the only case that touches engine code, and it’s a localised, well-defined extension point.

8.2 What’s real today vs what’s upstream

Be honest and precise about maturity — it reads as confidence, not weakness:

- **Real and working:** the rule engine (all 8 packs, scoring, verdict, two-tier), the FastAPI service, the full mock pipeline (offline photo→verdict), the CLIs, the test suites, and the Flutter app talking to the backend over LAN.
- **Real but needs keys/installs:** the live Gemini extraction (needs an API key) and PaddleOCR/OpenCV (need installing on the host). The architecture is done; these are switched on by config.
- **Deliberately upstream / future:** a real computer-vision symbol detector (today symbols come from the LLM), persistence (Postgres), authentication/RBAC, and rendered PDF/DOCX inspection reports. `requirements.txt` lists these as commented, planned dependencies.

8.3 Scaling story (for the “will it scale?” question)

The engine is stateless and pure-function, so it scales horizontally — run as many API workers as you like behind a load balancer; there’s nothing to coordinate. The expensive, latency-bound parts are OCR and the Gemini call, which is exactly why they’re already dispatched off the event loop via `run_in_threadpool` and why the model singleton avoids re-loading PaddleOCR per request. The natural next steps are a task queue (Celery/Redis) for heavy batch scanning and a Postgres store for scan history and the “crowdsourced enforcement” angle — both additive, neither requiring an engine change.

9. Interview & judging Q&A prep

Each answer is written to be said out loud in 20–40 seconds. The bold line is your headline; the rest is backup if they push. Answers are grouped by theme.

9.1 Architecture & the core idea

Q: Explain your project in one sentence. **A:** You photograph a product label; we tell you which legal declarations under the Legal Metrology Rules and FSSAI regulations are missing or wrong, citing the exact rule for each finding. The trick is that AI only *reads* the label and a deterministic engine *judges* it — so every verdict is auditable and tied to the law.

Q: Why separate “extraction” from “judging”? Why not one model that does both? **A:** Because they have opposite requirements. Reading a messy, multilingual, curved label is a fuzzy perception problem — perfect for an LLM. Deciding compliance is a legal problem that must be exact, reproducible, and explainable. **If an LLM decided compliance, you could never fully trust or audit the verdict, and you couldn’t defend it in an enforcement action.** By confining the LLM to extraction and letting deterministic code judge, the same label always yields the same verdict, and every finding cites a clause. It also means we can improve the reader and the rules independently.

Q: What’s the interface between the two halves? **A:** A single normalised JSON object we call the scan-input contract. The pipeline produces it; the engine consumes it. Its fields are keyed by declaration id — the same ids the rule packs use — which is what makes rules and extracted values line up with no glue code. Because the contract is explicit, we can swap Gemini or PaddleOCR for anything else without touching the engine.

Q: Walk me through what happens when I take a photo. **A:** (Give the Section 5 trace, compressed:) OCR reads word boxes for geometry; Gemini reads structured fields and the category; a reference object in frame calibrates pixels to millimetres; fusion merges them into the scan-input; the engine selects the applicable rule packs by category, runs each check, and returns a verdict, a 0–100 score, and violations each citing their rule.

9.2 The “why not just use AI / a trained model?” challenge

This is the teammates’ proposal and the single most likely hard question. Have the crisp version ready.

Q: Your teammates suggested putting the rules in a CSV and training the AI on them, or fine-tuning your own model. Why didn't you? A: Three separate ideas, and each is weaker than what we have. A CSV is just a less capable JSON — it can't express nested checks, per-rule severities, conditions, or citations, so it's a downgrade of the same "rules as data" idea. "Store in a database and let the AI retrieve by category" — we already select rules by category; that's a plain lookup, not AI, and files versus a database is just storage, not intelligence. And *fine-tuning a model to decide compliance* throws away our biggest advantages: you lose the exact citation, you lose reproducibility, verdicts become probabilistic and unexplainable, and you'd need thousands of hand-labelled compliant/non-compliant examples we don't have and can't easily get. **Machine learning belongs on the reading side — classifying the product, detecting a logo — never on the judging side.** We deliberately kept the legal decision deterministic.

Q: But isn't a rule engine less "smart" than AI? A: For this problem, deterministic is the smart choice. Compliance is defined by law, not by patterns in data — there's a correct answer written in a gazette. A rule engine encodes that answer exactly and can cite it. An ML model would only *approximate* it, less reliably, with no citation. We use AI precisely where intelligence helps — reading a hard image — and rules where correctness is non-negotiable.

Q: What happens when the law changes? Doesn't a rule engine go stale? A: The opposite — this is where it shines. A rule is a JSON entry with its citation. A gazette amendment is a JSON edit plus a POST /reload; it takes effect with no code change, no redeploy, and no retraining. A fine-tuned model would need re-labelling and re-training for the same change. Our packs even carry a version and effective date so you can stage an amendment before it goes live.

9.3 The rule engine internals

Q: How is a single rule represented? A: As a declaration inside a JSON pack: an id, a human label, a verbatim legal_reference, a severity (critical/major/minor), a condition for when it applies, and a list of typed checks — presence, format (a regex), value (a named validator), placement (which panel), font_height, or symbol. The engine reads this data; it isn't hard-coded logic.

Q: How do you compute the score and the verdict? A: The verdict is pure severity logic: any failed critical check makes it non-compliant; else any failed major makes it needs-review; else compliant — minor failures never downgrade past compliant. The score is a weighted pass ratio: weighted passes over weighted (passes + fails), with weights critical 3, major 2, minor 1. Checks we couldn't evaluate are SKIPPed and excluded from both — so a label is never penalised for something we couldn't see.

Q: What's the difference between PASS, FAIL, and SKIP? A: PASS and FAIL both count toward the score; SKIP doesn't. SKIP means "not applicable or couldn't evaluate" — for example, a font-height check when no glyph height was measured, or a value check on a field that wasn't extracted. Crucially, if a required field is missing, its *presence* check FAILs once, and its other checks SKIP — so we report the root cause once instead of double-penalising.

Q: How do packs combine for a given product? A: The Legal Metrology base pack always applies. Category packs apply when the product's category matches their applies_when list, and they stack. The loader sorts base-first and merges declarations by id — so a category pack can override a base rule by reusing its id — and merges reference standards de-duped by id. The engine only ever sees the merged ruleset.

Q: What if a rule's condition references something the engine doesn't know? A: Unknown condition tokens evaluate to True — a deliberate fail-safe, meaning "check it anyway" rather than silently skipping a legal requirement. The mirror invariant is that every context flag a pack actually relies on is a registered flag defaulting False, kept in sync across the engine model, the prompt, and the API schema — so narrow rules stay dormant until the pipeline positively detects the trait.

9.4 The pipeline, OCR, and the vision-LLM

Q: Why do you use OCR *and* an LLM? Isn't that redundant? **A:** They do different jobs. The LLM is excellent at *what the text says and which declaration it is*, but unreliable at *precise pixel geometry*. Rule 7(2) mandates a minimum letter height in millimetres, so we need real glyph pixel heights — that's what OCR bounding boxes give. So the LLM tells us the net quantity is "500 g"; OCR tells us how tall those numerals are. Complementary, not redundant.

Q: How do you measure letter height in millimetres from a photo? **A:** A photo has no inherent scale, so we put a known-size object in the frame — typically a credit/ID card at 85.6 mm wide, or an ArUco marker. Calibration computes millimetres-per-pixel from that reference. Then for a declaration we take the OCR words that make it up, take the *median* of their pixel heights — median so one bad box doesn't skew it — and multiply by the scale. If there's no reference or no matching words, we return no height and the font-height check SKIPS rather than guessing.

Q: How do you stop the LLM from hallucinating a value? **A:** Three ways. We call it at temperature 0 with a JSON-only response type for determinism. The prompt instructs it to omit declarations that are genuinely absent rather than invent them, and to lower confidence when unsure. And — most importantly — even if it did misread, it can only produce a *value*; the deterministic engine still judges that value against the rule and cites the law, so a misread becomes a checkable field, not a false verdict.

Q: The prompt — is it hard-coded? **A:** No, it's generated from the live rule packs. We pass Gemini exactly the declaration ids and symbol names the currently-loaded packs define, so when we add a pack, the extractor automatically starts asking for its fields. The reader can never drift from the rules.

Q: What if there's no internet, no API key, or the OCR library isn't installed? **A:** Every heavy layer has a deterministic mock and auto-detects: no key or no library falls back to mock instead of crashing, so the whole photo→verdict path runs offline — which is also how we demo reliably. And if real OCR is installed but fails on a specific image, we degrade to no word boxes and continue on the LLM alone; only the font-height check SKIPS. A broken dependency degrades one check; it never 500s the scan.

9.5 Robustness and edge cases

Q: What if I photograph something that isn't a label — a book, my face? **A:** There's a guard for exactly that. Before judging, we check whether the image plausibly is a packaged-commodity label — any extracted field, any detected regulatory symbol, any set context flag, or label-fingerprint text like "MRP", "net qty", "FSSAI", "batch no", or a weight/volume. If none of those are present, we return a distinct NO_LABEL verdict — "no packaged-commodity label detected" — instead of scoring a book as a catastrophically non-compliant product. It's an honest outcome, not a false failure.

Q: How do you handle the dozens of date formats on labels? **A:** The date parser is deliberately forgiving — it extracts every plausible date from messy text (dd/mm/yyyy, mm/yyyy, mm-yy, "Mon YYYY", and so on), normalises two-digit years into the 2000s, and clamps to a sane year range. Then the validators reason over the parsed dates — a manufacture date can't be in the future; an expiry must be after manufacture. We extract liberally and validate strictly, rather than demanding one rigid format the printer may not have used.

Q: A malformed rule pack or a bad regex — does it crash? **A:** No. `from_dict` reads pack fields defensively with defaults, a bad regex in a format check SKIPS with an explanatory detail instead of throwing, and the health and reload endpoints eagerly load packs so a malformed pack surfaces its error at deploy time, not mid-scan.

9.6 The two-tier / reference standards

Q: There are things a photo can't verify — composition, contaminants. Do you just ignore them? **A:** No — and this is a point of pride. We split requirements into two tiers. Photo-verifiable

ones are scored and cite their rule. Lab-only ones — composition minima, additive limits, heavy-metal caps — are surfaced as “applicable standard; verify in lab” but are never scored. They live on a separate data path and literally can’t affect the number, so our score only ever reflects what we could actually see. That’s more honest, and more useful to an inspector, than pretending a camera can measure milk fat.

Q: How do you *guarantee* those don’t affect the score? A: It’s structural, not a promise. The score comes only from check results; the verdict only from violations; and those are created in exactly one place, iterating the scored declarations. Reference standards are a different data type that never becomes a check — there’s no code path from one to the other. A test recomputes the score to prove it, so the guarantee can’t silently regress.

9.7 Legal accuracy & trust

Q: Are your legal citations real? A: The clause numbers and the Rule 7(2) font-height table values are transcribed verbatim from the 2011 Rules as amended — notably G.S.R. 629(E) of 2017 — and we note the source in the pack. To be precise: the *citations* are real; the *regexes* that detect each provision are our own engineered detectors, not gazette text. We never invent clause numbers, and each pack carries a source note pointing to the official gazette to verify before a live deployment.

Q: Why should an inspector trust this over their own judgement? A: It’s not a replacement for judgement — it’s an accelerator that shows its work. Every finding names the exact rule and gives a human-readable reason; every check we ran is in the report, including passes and skips; and lab-only matters are flagged for follow-up. The inspector can audit each finding against the cited clause. It makes them faster and more consistent, and it leaves a defensible paper trail.

9.8 Scaling, security, production

Q: Will this scale to millions of products? A: The engine is stateless and pure, so it scales horizontally — add API workers behind a load balancer, nothing to coordinate. The latency-bound parts are OCR and the Gemini call, which we already run off the event loop in a threadpool, with the OCR model loaded once as a singleton. For high volume, the additive next steps are a task queue for batch scans and a Postgres store for history — neither touches the engine.

Q: What about security and multi-user access? A: Today CORS is open for local development, with a note to lock it to the deployed origins before production. Authentication, role-based access for officers versus consumers, and a database are on the roadmap and listed as planned dependencies — deliberately deferred so the hackathon build proves the core: the engine and the pipeline.

9.9 Curveballs & honest weaknesses

Answering these calmly signals maturity. Never bluff.

Q: What’s the weakest part of the system right now? A: The reading accuracy on genuinely poor photos, and symbol detection. Today symbols come from the LLM rather than a dedicated computer-vision detector, so logo recognition is best-effort. That’s a known, scoped gap — the engine already treats symbols through a registry, so we can drop in a real detector keyed by the same names with no rule changes. I’d rather be precise about that than overclaim.

Q: If the LLM misreads a value, doesn’t your verdict become wrong? A: It can affect a single field, yes — extraction is the fuzzy part. But the design contains the blast radius: a misread produces a wrong *field*, which the engine still checks transparently and reports with its reasoning, so a human can see and correct it. The officer can override context and re-scan. What can’t happen is a hallucinated *verdict*, because the LLM never decides compliance.

Q: Isn’t maintaining rules by hand a lot of work? A: It’s deliberate, bounded work done once per regulation by someone who can read the gazette, and it’s why the output is trustworthy. Auto-scraping legal PDFs gets clause numbers and provisos wrong, and one wrong citation makes an

inspector distrust the whole tool. Encoding by hand with the exact reference is the price of an auditable verdict — and after that, updates are small JSON edits, not re-scrapes.

Q: What would you build next with more time? A: A dedicated CV symbol detector, persistence with scan history to enable the crowdsourced-enforcement angle (consumers flagging suspect labels), authentication and officer roles, and rendered PDF inspection reports for the paper trail. The architecture already has clean seams for all four.

Q: Why Python for the engine — isn't it slow? A: The engine's work is a few dozen regex and comparison operations per scan — microseconds; the latency is entirely in OCR and the network call to Gemini. Python's readability makes the *legal logic* easy to audit, which matters far more here than raw speed, and the engine being pure-stdlib is a bigger reliability win than a faster language would be.

10. One-page cheat-sheet (the night before)

The one-liners (memorise verbatim)

- **The pitch:** “Photograph a label; we tell you which legal declarations are missing or wrong, citing the exact rule. AI reads the label, a deterministic engine judges it.”
- **The principle:** “AI extracts, rules judge.” Extraction is fuzzy perception (LLM's strength); judging is exact law (code's job).
- **Why not train an AI to judge:** “You'd lose the citation, lose reproducibility, and need thousands of labelled examples. ML belongs on the reading side, never the judging side.”
- **Rules as data:** “Every rule is JSON with its verbatim citation. A law change is a JSON edit plus /reload — no code, no retraining.”
- **Two-tier:** “Photo-verifiable checks are scored; lab-only standards are surfaced as ‘verify in lab’ on a separate path that structurally cannot touch the score.”
- **Honesty:** “Clause numbers are transcribed verbatim from the gazette; the regexes that detect them are our own.”

Key numbers to have ready

- **8 rule packs** auto-loaded (1 base + 7 FSSAI category packs).
- **Severity weights:** critical **3**, major **2**, minor **1**.
- **Score** = $100 \times \text{weighted PASS} \div \text{weighted (PASS + FAIL)}$; SKIP excluded; 100.0 if nothing scored.
- **Verdict:** any failed critical → non_compliant; else any failed major → needs_review; else compliant.
- **19 context flags**, all default False, mirrored in 3 files.
- **Card reference** = **85.6 mm** wide for pixel→mm calibration.
- **Rule 7(2)** = the minimum-letter-height requirement (font-height check).
- **Gemini** at **temperature 0.0**, JSON-only response.
- **CLI exit codes:** 0 compliant / 1 needs-review / 2 non-compliant.

The file map (backend/)

Path	Role
rule_engine/models.py	Dataclasses: Check, Declaration, ReferenceStandard, Pack, RuleSet, ScanInput, ComplianceReport, ScanContext (19 flags)
rule_engine/loader.py	Loads rulepacks/*.json; build_ruleset(category) selects + merges packs

Path	Role
rule_engine/checks.py	VALIDATORS registry, KNOWN_SYMBOLS, condition_met (unknown→True), evaluate_check dispatcher
rule_engine/engine.py	evaluate: check loop, _looks_like_label guard, _verdict, _score, _collect_reference_standards
rulepacks/*.json	The 8 rule packs — the actual law, as data
pipeline/ocr.py	PaddleOCR word boxes; degrades to empty on failure
pipeline/gemini.py	Vision-LLM field extraction; rule-driven prompt
pipeline/calibration.py	Reference object → mm-per-pixel
pipeline/fusion.py	Merge OCR + LLM into the scan-input; median glyph height
pipeline/pipeline.py	Orchestrator: extract_scan_input, extract_and_evaluate
app/main.py	FastAPI endpoints, @lru_cache packs, run_in_threadpool
app/schemas.py	Pydantic boundary; ContextIn mirror; ReportOut.reference_standards
run_scan.py / extract.py	CLIs: judge / read (+ --evaluate for both)

The 8 rule packs

Pack	Scope	Declarations	Reference standards
legal_metrology_2011	base (always)	10	0
fssai_food_2020	category	9	0
fssai_packaging_labelling_2011	category	8	0
fssai_contaminants_2011	category	0	5 (pure Tier-2)
fssai_organic_2017	category	3	1
fssai_fortification_2018	category	3	1
fssai_nutraceutical_2016	category	4	2
fssai_alcoholic_2018	category	11	3

The check types

presence (is the field there) · format (regex match) · value (named validator, e.g. positive_currency, date logic, fssai_14_digits) · placement (correct panel) · font_height ($\geq$ Rule 7(2) minimum for the panel area) · symbol (a known regulatory mark is present).

Mini-glossary

- **Declaration** — one legal requirement (e.g. “MRP must be printed”), with id, citation, severity, condition, checks.
- **Check** — one testable assertion inside a declaration; produces a CheckResult.
- **CheckResult** — outcome PASS / FAIL / SKIP for one check.
- **Violation** — a FAILED check, carrying its severity and legal reference.
- **ReferenceStandard** — a Tier-2, lab-only requirement; never scored.
- **ScanContext** — the 19 boolean traits (e.g. is_imported, has_aspartame) that gate narrow rules.
- **ScanInput** — the normalised contract the pipeline hands the engine.
- **RuleSet** — the merged packs for one product category.

- **Pack** — one JSON file of declarations + reference standards for a regulation.

If you blank on a hard question

- **“Why not AI for everything?”** → “AI can’t cite the law or reproduce a verdict. It reads; rules judge.”
- **“Is it accurate?”** → “The judging is deterministic and exact. The reading is the fuzzy part — and even a misread only produces a checkable field, never a false verdict.”
- **“Weakest part?”** → “Symbol detection is LLM-based today, not a dedicated CV detector — a scoped, known gap with a clean seam to fix it.”
- **“Does it scale?”** → “Engine is stateless and pure — scales horizontally. Only OCR and the Gemini call are latency-bound, and those already run off the event loop.”